Section 1

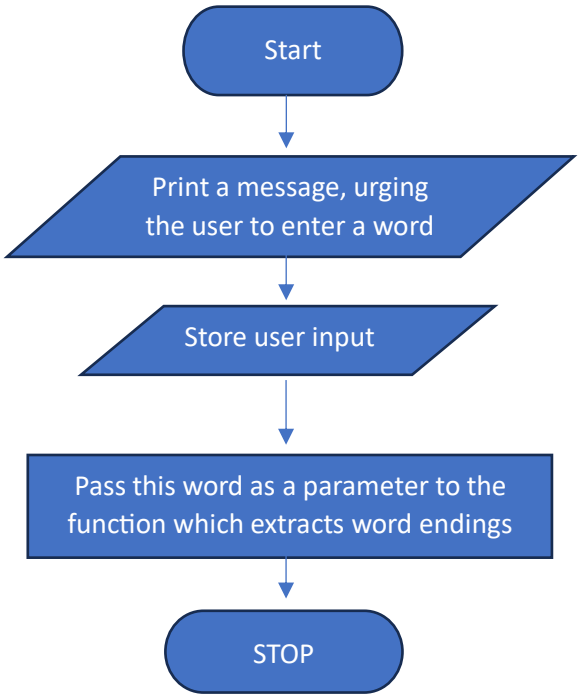
Essence of solution

This poetry assistant adopts a phonetic approach to finding rhymes for a word entered by the user. My algorithm finds the position of the last vowel in the input word, then it finds the first occurrence of a consonant/blank space before this last vowel. All the letters after this consonant/blank space are extracted as the word ending. For example, the ending of the word 'algorithm', 'word' and 'ending' and 'eat' are extracted as 'ithm' 'ord', 'ing' and 'eat' respectively. This algorithm also takes into consideration words that end with a consonant followed by a single vowel. To prevent the algorithm extracting a word ending of only a single vowel, my algorithm checks if the last letter is a vowel. If so, it finds the position of the second last vowel instead of the last vowel, then it finds the first consonant/blank before the second-last vowel. All the letters after this consonant/blank are extracted. For example, the ending of 'essence', 'case' and 'fleece' are 'ence', 'ase' and 'eece' respectively, instead of just 'e'. Once the word ending is extracted, it is compared to the ending of all the words in the list of words, which are stored in a vector data structure. If a match is found that word is pushed into the dynamic array of rhyming words. Once all the words in the list have been traversed, the dynamic array is printed to the console and the user interface. If the word entered by the user does not contain a single vowel, a message is printed indicating that it is not a valid word. If the dynamic array of rhymes is empty, a message is printed stating that no rhymes were found. Additionally, the user interface contains a button that allows the user to enter a new word, so the algorithm executes again.

Section 2: Original algorithms

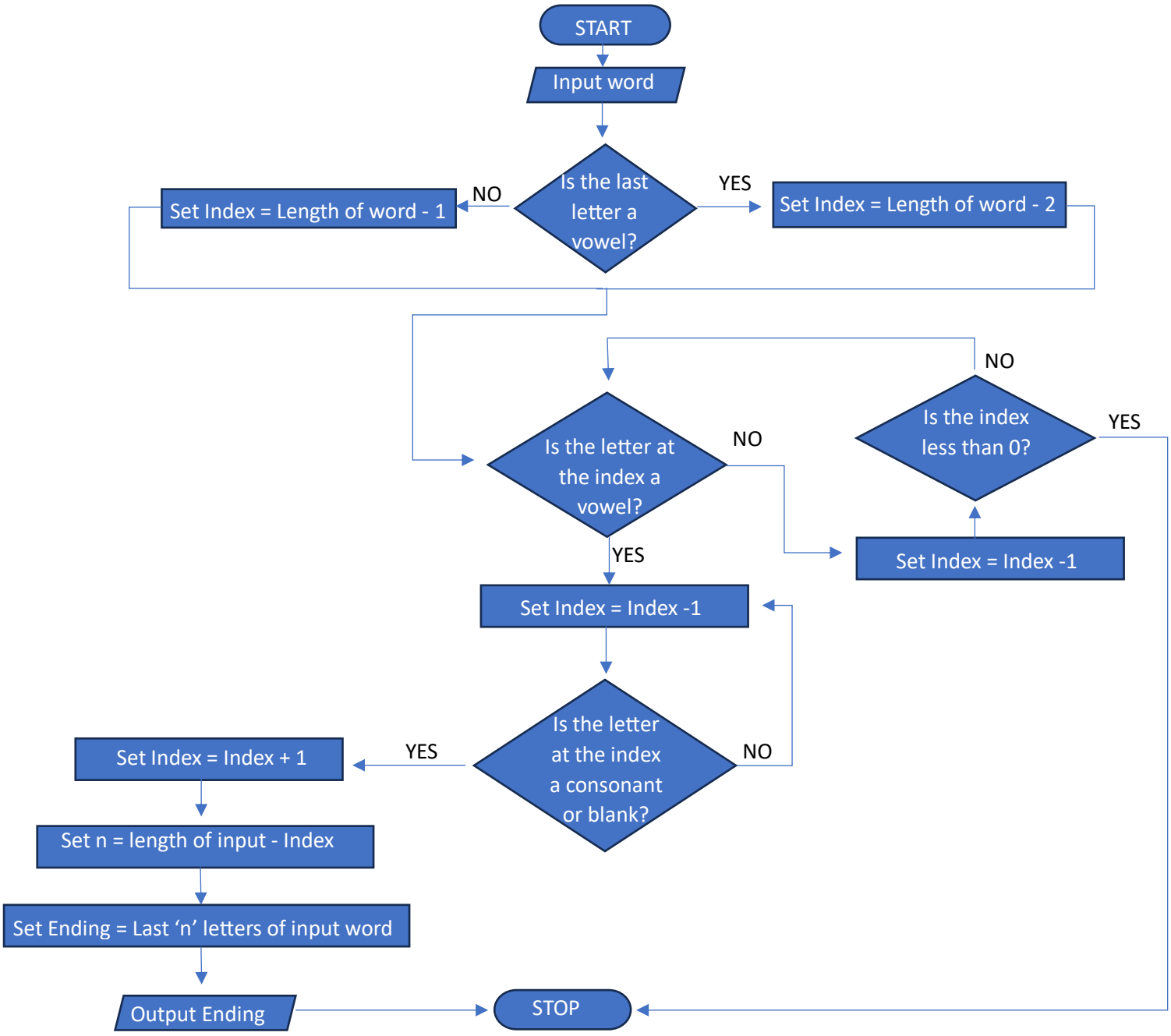
The original algorithms in my solution are interconnected. Firstly, when the user clicks the button for a new word, they are prompted to enter a word. The original algorithm for this function is shown as a flowchart in section 2.1. Secondly, the original algorithm in section 2.2 extracts the ending of the word obtained through algorithm 2.1. Furthermore, algorithm 2.3 uses the word ending extracted by algorithm 2.2 as input, to search for and print rhymes with the same ending.

2.1: Original algorithm: When the button for a new word is clicked



2.2: Original algorithm: extracting word ending.

This algorithm takes the word entered by the user as input. If the word ends with a vowel, it sets a variable index to the index of the second-last letter. If not, the variable index is set to the position of the last letter. Then, the algorithm searches for the first occurrence of a vowel before this index. Once a vowel is found, the algorithm searches for the first occurrence of a consonant/blank space before this vowel. The index is then increased by 1 so that it contains the position of the first vowel after this consonant/blank space. Then, the index is subtracted from the length of the word, to obtain the number of letters 'n' after the consonant/blank space. The last 'n' letters of the word are returned as the word ending.

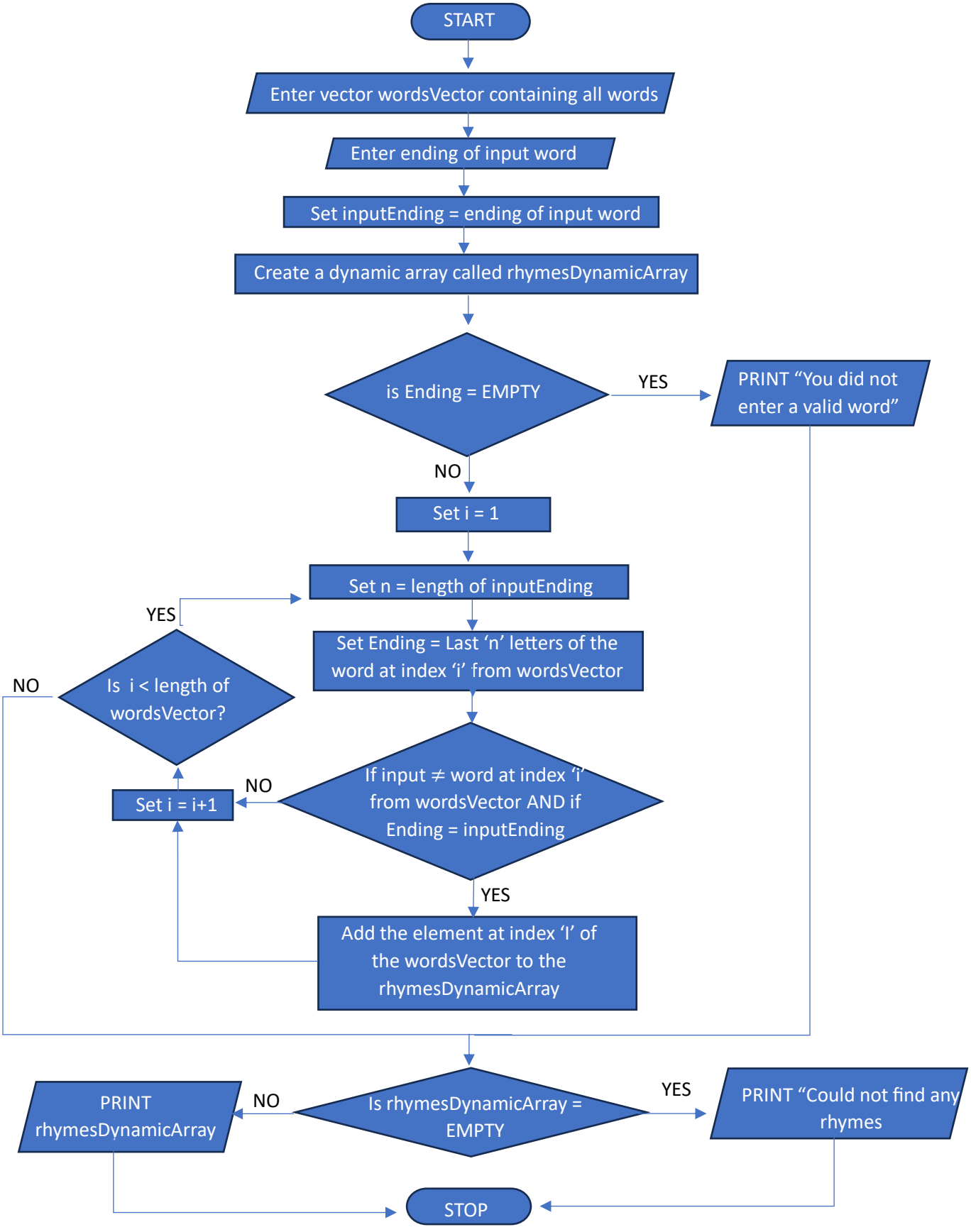


2.3: Original algorithm: searching for rhymes.

This algorithm takes two inputs: the vector containing the list of words, and the ending of the input word extracted through algorithm 2.1 ‘inputEnding’. An empty dynamic array is created to store rhyming words. If the variable inputEnding is empty, this indicates that the word entered by the user does not contain any occurrence of a vowel, since no ending could be extracted. Therefore, a message is printed stating that the user did not enter a valid word. However, if the word ending exists, a vector containing the list of words is traversed, starting at an index i = 1, till when the index reaches the length of the vector. A variable ‘n’ stores the length of the input word ending. This variable ‘n’ is used to compare the last ‘n’ letters of each word in the word list to the inputEnding. If a match is found, the word at the index ‘i’ in the vector is pushed to the dynamic array created for storing rhymes. When the index ‘i’ reaches the length of the vector, all of the words have been traversed, the rhymes dynamic array is printed. If the dynamic array is empty, a message is printed stating that no rhymes were found.

Flowchart given on next page.

2.3 Original algorithm: searching for rhymes.



Section 3: Pseudocode

3.1 Pseudocode for the algorithm which is executed when the button for a new word is clicked

```
//Callback function for when the new word button is clicked
NEW-WORD()
```

1. Input = prompt for user input
2. SEARCHING-RHYMES(input) // The SEARCHING-RHYMES function is called, passing the input word as a parameter

3.2 Pseudocode for original algorithm that extracts word endings.

```
//Function to extract the ending of the input word.
EXTRACT-ENDING(INPUT)
```

1. **if** INPUT[INPUT.length-1]≠ “a” and INPUT[INPUT.length-1]≠ “e” and INPUT[INPUT.length-1]≠ “i” and INPUT[INPUT.length-1]≠ “o” and INPUT[INPUT.length-1]≠ “u”
2. index = INPUT.length – 1 //index of last letter
3. else
4. index = INPUT.length – 2 //index of second-last letter
5. **while** index ≥ 0
6. //This conditional statement finds the last vowel (second-last if the last letter is a vowel)
7. **if** INPUT[index] = “a” or INPUT[index] = “e” or INPUT[index] = “i” or INPUT[index] = “o” or INPUT[index] = “u”
8. //Once the last/second-last vowel is found, the index is decreased by 1, in order to check the letter before it
9. index = index – 1
10. //If the letter at this index is not a consonant, the index value is decremented again. This condition continues iterating as long as the letter at the given index is not a consonant/blank space.
11. **if** INPUT[index] = “a” or INPUT[index] = “e” or INPUT[index] = “i” or INPUT[index]= “o” or INPUT[index] = “u”
12. index = index – 1
13.
14. //the index has reached a position where there is a consonant/blank space
15. //Incrementing the index by 1 so that it is at the first vowel after the consonant/blank.
16. index = index + 1
17. //The variable n stores the number of letters in the word ending.
18. n = INPUT.length – index
19. Ending = last *n* elements of INPUT
20. **return** Ending
21. index = index - 1

3.3 Pseudocode for the original algorithm that finds and prints rhymes

```
//Function to find and print rhyming words.
//this function takes two inputs: the word entered by the user and the vector containing the list of words
SEARCHING-RHYMES (INPUT, wordsVector)
1.  //storing the input ending returned by the EXTRACT-ENDING function.
2.  inputEnding = EXTRACT-ENDING(INPUT);
3.  //creating a dynamic array to store rhyming words
4.  new dynamic array d
5.  d[0]=0 //storing the length of the dynamic array in index 0 of the array
6.  if inputEnding is empty
7.          print "You did not enter a valid word"
8.  else
9.          for i=1 to wordsVector[0] // wordsVector[0] contains the length of the vector
10.         n = inputEnding.length
11.         Ending = last 'n' letters of wordsVector[i]
12. //conditional statement that pushes words from the word list to the dynamic array containing
    rhymes, if the ending of the word and input word match
13. if input≠wordsVector[i]
14.         if Ending = inputEnding
15.                 d[0] =d[0]+1//increasing the length of the dynamic array by 1
16.                 d [LENGTH[d]]=(wordsVector[i]); //storing the rhyme in the last element of the
                                                    dynamic array
17. if d[0]=0 //d[0] contains the length of the dynamic array
18.         print "Could not find any rhymes."
19. else
20.         print d
```

Section 4: Choice of data structures

4.1: Storing the list of words in a vector

Data Structure	Suitability	Reason
Vector	Suitable	Among the data structures listed here, a vector is most suitable for storing the word list for a number of reasons: 1. Accessibility: any element stored in a vector is accessible in any order. This feature is required as my algorithm needs to access every word in the vector in order to find rhymes. 2. Fixed length: A vector always has a fixed number of elements, therefore, the fixed number of words in the word list can easily be stored in a vector. 3. Cannot add/remove elements: My algorithm does not need to add or remove any elements or alter the data structure. It only accesses the elements to check their ending.
Queue	Unsuitable	In a queue, only two elements are accessible: the back of the queue for enqueuing elements, and the front of the queue for dequeuing elements. For the purpose of comparing each element in the queue to the word ending, I would have to dequeue an element with each comparison, in order to access the next element. This would cause the entire data structure to be empty by the time every word in the list has been examined. The queue must be filled again with all the words from the list in order to search for another word. Alternatively, every element removed from the queue could be added to another queue. However, both these processes introduce redundancy, as the entire data structure must be filled in again. Since the algorithm only needs to make comparisons to the elements, it is counter-intuitive to alter the data structure every time a comparison is made.
Stack	Unsuitable	Only the top element of a stack is accessible. A stack would be unsuitable for a similar reason that makes a queue unsuitable. Each comparison to an element would entail popping an element from the stack, in order to make the next element accessible. The elements popped from one stack could be pushed to another stack, but this would take extra space and time which is not required
Dynamic Array	Unsuitable	A dynamic array is extensible, elements can be added or removed from it. However, as the length of the word list is constant, this extra functionality is not required.

4.2: Storing rhymes in a dynamic array

Data Structure	Suitability	Reason
Dynamic Array	Suitable	A dynamic array is most suitable for the task of storing rhymes for the following reasons: 1. Extensible: Elements can be added to a dynamic array. My algorithm compares the ending of the input word one by one to all the words in the list. Whenever a new rhyme is found, it can easily be added to a dynamic array. **2.** Accessible: All the elements of a dynamic array are accessible. Hence, the entire dynamic array of rhymes can be printed at once, without having to print each rhyme individually.
Vector	Unsuitable	The algorithm compares the word ending of each word one by one to the ending of the input word. Whenever a match is found that word must be added as an element to the data structure which stores rhymes. A vector is not extensible, and elements cannot be added to it. Using a vector, each time a new rhyme is found, a new vector must be made with a different length, which wastes time. Therefore, a vector is unsuitable for this task.
Queue	Unsuitable	Stacks and queues are extensible; whenever a new rhyme is found, it can be enqueued in the queue or pushed to the stack. However, elements can only be dequeued from the front of the queue or popped from the top of the stack. Therefore, using a stack/queue, the rhymes are printed one at a time. These data structure could be considered, however, a dynamic array is preferred as all the elements are accessible and printed at once.

Section 5

(Source code and video provided in the folder)

Section 6
Consideration of defects and potential solutions

Defects	Solutions
My algorithm design finds all the words from the list that have the same ending, even if this ending is preceded by vowels that change the rhyme. For example, for the word 'case', the word 'please' is returned among the rhymes. Although 'please' does have 'ase' in the ending, the 'e' preceding it changes the pronounciation entirely.	This problem could be solved by adding an extra conditional statement within the one that checks for the same word ending. This additional conditional statement should ensure that words with the same ending are returned provided that the ending is not immediately preceded by a vowel.
My algorithm is incapable of differentiating word endings that have the same spelling but different pronounciation. For example, the word 'cough' is returned as a rhyme for 'through'	The problem could be solved if the algorithm searches a list of words with phonetic spellings instead of alphabetical spellings.